

HPCプログラミングセミナー

GPUプログラミング入門（OpenACC編）

第2講：OpenACCでのGPU実装と最適化基礎

高度情報科学技術研究機構

Research Organization for Information Science and Technology (RIST)

質問は随時受け付けます

E-mail (hpc-seminar@rist.or.jp) もご利用ください

この資料は、HPCプログラミングセミナー「アクセラレータ入門」として実施していたものと同じ内容です
OpenACCを用いたGPUプログラミングの基礎を解説するという資料の主旨を明確にするため、タイトルを変更しました

本講の目的

- 第1講で解説したGPUアーキテクチャの基礎を元に、OpenACCを用いてアプリケーションをGPUに実装して最適化するための基礎を身につける

自学自習のために

■ サンプルコードも用意（ご自由にお使いください）

- ▶ OpenACCを中心に、OpenMP Target, Kokkosのサンプルを含む
- ▶ HPCIポータルサイトから入手可能
 - ▶ https://www.hpci-office.jp/events/seminars/seminar_texts

おことわり

- 本講はOpenACCの実装方法の内、代表的な物のみ取り上げています（詳細の把握: 補足と参考文献の[1]などを参照）
- NVIDIA HPC SDKの利用を前提に説明しています
- 基礎ですが一部発展的な内容も載せています
 - ▶（発展）と記しています（飛ばして構いません）
- 個別の事例については別途相談頂くか、HPCIヘルプデスクを通じてお問い合わせください
 - ▶ 現在のHPCIの利用の有無は関係ございませんのでお気軽にどうぞ

<http://www.hpci-office.jp/pages/helpdesk/>

GPU向けコードの実装方法

■ CUDA, OpenCL

- ▶ ユーザがGPU向けコードを記述
- ▶ 低レベルな方法で、柔軟性高く、高性能なコードを作成可能
- ▶ CPU向けコードとは別に作成する必要がある

cf. OpenMP Target

- OpenACCと同様な方法でGPU向けコード作成が可能。「補足: OpenACCとOpenMPの比較」を参照

■ OpenACC

- ▶ 指示文挿入によりコンパイラがGPU向けコードを生成
- ▶ CPU向けコードとの共通化が可能

■ GPU対応ライブラリ

- ▶ 「計算処理」自体をライブラリ提供の関数で置き換える
 - ▶ 目的に合致したものが存在する場合は容易に高い性能を得ることが可能
- ▶ 移植性を保つようなライブラリも存在: Thrust, Kokkos

Thrust: <https://developer.nvidia.com/thrust>
Kokkos: <https://github.com/kokkos/kokkos>

性能と手間のバランスが取れているOpenACCを紹介

- 後ほど他2つを学習する上でも有用 (学習内容、リソース)
- 以上3手法のハイブリッド構成での実装も可能

OpenACC

- 規格は共同で策定され、一般に公開されている
 - ▶ ベンダ、メーカ、大学、研究機関らが参画
 - ▶ <https://www.openacc.org/specification>
- 規格に応じて、メーカやコミュニティがコンパイラをリリース
 - ▶ NVIDIA, GNU, LLVMなどが対応
- 本講はOpenACC 2.7を扱う
 - ▶ 最新（2026年3月時点）は3.4.
 - ▶ 規格書は公式サイトからダウンロード可能
- 2.7は主要な機能を含む
 - ▶ 基礎として十分
 - ▶ 参考文献や先行事例も多く、自学自習にも最適
 - ▶ まずは2.7（or 2.6）を学習してから先に進むことを推奨

OpenACCによる実装例：SAXPY (C)

OpenMP (CPU向け)

```
void saxpy(int n,  
           float a,  
           float *x,  
           float *restrict y)  
{  
    #pragma omp parallel  
    #pragma omp for  
    for(int i=0; i<n; ++i) {  
        y[i] = a*x[i] + y[i];  
    }  
}  
  
...  
saxpy(1024, 2.0, x, y)  
...
```

OpenACC

```
void saxpy(int n,  
           float a,  
           float *x,  
           float *restrict y)  
{  
    #pragma acc parallel  
    #pragma acc loop  
    for(int i=0; i<n; ++i) {  
        y[i] = a*x[i] + y[i];  
    }  
}  
  
...  
saxpy(1024, 2.0, x, y)  
...
```

OpenMP (CPU向け)とほぼ同じスタイル、挿入の分量で実装出来る

OpenACCによる実装例 : SAXPY (Fortran)

OpenMP (CPU向け)

```
subroutine saxpy(n, a, x, y)
  real(4) :: x(n), y(n), a
  integer :: n, i

  !$omp parallel
  !$omp do
    do i=1,n
      y(i) = a*x(i) + y(i)
    end do
  !$omp end do
  !$omp end parallel
end subroutine saxpy
...
call saxpy(1024, 2.0, x, y)
...
```

OpenACC

```
subroutine saxpy(n, a, x, y)
  real(4) :: x(n), y(n), a
  integer :: n, i

  !$acc parallel
  !$acc loop
    do i=1,n
      y(i) = a*x(i) + y(i)
    end do
  !$acc end parallel
end subroutine saxpy
...
call saxpy(1024, 2.0, x, y)
...
```

C, Fortran共に同様のスタイル、少ない負担でGPU実装可能

OpenACCのメリット

■ 段階的な実装が可能

- ▶ 既存のCPU版コードに基づき実装ができる
 - ▶ 指示文を追加するスタイルのコーディング
 - ▶ 追加のたびに結果検証していくことが容易

■ 単一のコード

- ▶ 同じコードを異なるアーキテクチャ（例：Intel x86 CPU, NVIDIA GPU）上でコンパイル可能
 - ▶ GPUだけでなくCPUにも対応
 - ▶ コンパイラがアーキテクチャに合わせて並列化する
 - ▶ 既存コード（CPU向け開発版）を維持可能
 - ▶ OpenMP指示文との併記も可能（コンパイラが選択する）

■ 習得が容易

- ▶ HPCでよく使われる「C/C++, Fortran」 + 「指示文」のスタイル
- ▶ ハードの詳細については（CUDA程には）学習不要

OpenACCのデメリット

■ CUDAのような柔軟な実装・最適化は難しい

- ▶ GPUの実行モデルに合わせた最適化は難しい
 - ▶ GPUに合わせ過ぎるとCPU実行時に性能が落ちてしまう
- ▶ プロセッサ特有の機能を十分には活用できない
 - ▶ NVIDIA GPUにおけるシェアードメモリなど

■ 解決策の一つ

- ▶ 各プロセッサ向けの低レベルな実装方法（NVIDIA GPUでのCUDAなど）の部分的な利用
 - ▶ 例：OpenACC+CUDAのハイブリッド実装
 - ▶ メリットである「単一のコード」とは相反する方法。バランスが必要となる
 - ▶ 更新頻度の少ないコードに限ってCUDAで実装するなど

- それでは具体的にどうやって使うのかを見ていききたいと思います

OpenACCの記法

備考: 構文と同名の節に注意
(例: data構文とdata節)

■ 指示文, 節, 構文

- ▶ 指示文 (ディレクティブ) : コンパイラにGPU向けコード生成を指示
- ▶ 節 (クローズ) : 指示文の挙動を補足 (optional)
- ▶ 構文 (コンストラクト) : 指示文、および指示文で指定された処理 (ループや構造化ブロックなど) から構成

[C] **#pragma acc** directive-name [clause [,] clause]..]

[F] **!\$acc** directive-name [clause [,] clause]..]

directive-name : 指示文を指定
例 : kernels, parallel, data, loop

clause : 節を指定 (省略可)
例 : independent, copy, async

指示文の適用範囲 (1/2)

代表例を紹介
詳細は規格書を確認

Cは直後の構造化ブロック全てに、Fortranはendまでに含まれた部分

```
#pragma acc directive-name [clause [[,] clause]..]  
{  
    構造化ブロック  
}
```

C

例:

```
#pragma acc kernels loop  
for (int j=0; j<n; j++)  
{  
    ...  
}
```

```
#pragma acc data (...)  
{  
    #pragma acc kernels  
    for (int j=0; j<n; j++)  
    {  
        ...  
    }  
    ...  
}
```

```
!$acc directive-name [clause [[,] clause]..]  
    構造化ブロック  
!$acc end directive-name
```

Fortran

例:

```
!$acc kernels loop  
do j=1, n  
    ...  
end do  
!$acc end kernels loop
```

```
!$acc data (...)  
    !$acc kernels  
    do j=1, n  
        ...  
    end do  
    !$acc end kernels  
    ...  
!$acc end data
```

指示文の適用範囲 (2/2)

代表例を紹介
詳細は規格書を確認

直後の記述のみに働く場合

例

```
#pragma acc loop
for (int j=1; j<n-1; j++) { ←jループを対象に並列化
    for (int i=1; i<n-1; i++) {
        ...
    }
}
```

C

例

```
!$acc loop
do j=1, n ←jループを対象に並列化
    do i=1, n
        ...
    end do
end do
```

Fortran

例

```
#pragma acc loop collapse(2)
for (int j=1; j<n-1; j++) { 直下の入れ子になった2個
    for (int i=1; i<n-1; i++) のループまで並列化
        ...
    }
}
```

備考: collapse節については
補足を参照

例

```
!$acc loop collapse(2)
do j=1, n 直下の入れ子になった2個
    do i=1, n のループまで並列化
        ...
    end do
end do
```

備考: collapse節については
補足を参照

OpenACC 3種類の基本構文

GPUプログラミングにて必須な要素を表現するもの

■ Compute構文*

- ▶ デバイスにオフロードさせる領域の指定

*備考: OpenACC 2.5 ではAccelerator Compute構文と呼ばれていた

■ Data構文

- ▶ デバイス上データの処理位置と処理内容（確保, 転送, 解放）の指定
- ▶ 指定した領域（data region）において、データがデバイス上に存在することをコンパイラに教える

■ Loop構文

- ▶ 並列化するループを指定
- ▶ 特殊な並列化の指定（independent, reduction, private, etc.）
- ▶ 並列化の粒度と階層の設定（発展）

Compute構文

■ デバイスにオフロードさせる領域を指定する構文（必須）

- ▶ kernels構文, parallel構文, serial構文の3種類
 - ▶ GPUデバイスで並列処理する構文はkernelsとparallel
 - ▶ Compute構文で指定された構造化ブロックはCompute regionと呼ばれる
 - ▶ kernelsの場合はkernels region、parallelの場合はparallel regionとも呼ばれる

■ kernels構文のみで多くのループをGPUへオフロード可能

- ▶ data構文とloop構文に相当する命令はコンパイラが補完
 - ▶ 正しく補完されたかはユーザが確認する
(NVIDIA HPC SDK: -Minfo=accelオプションを指定してコンパイル)

kernels構文のみでの動かし方をまずは見てみましょう

kernels構文でのGPU実装：SAXPY

```
3 void saxpy(int n,  
4           float a,  
5           float *x,  
6           float *restrict y)  
7 {  
8   #pragma acc kernels  
9   for(int i=0; i<n; ++i){  
10      y[i] = a*x[i]+y[i];  
11   }  
12 }  
13 int main(int argc, char **argv)  
14 {  
15   int N = 1024;  
16  
17   float *x = (float*)malloc(N * sizeof(float));  
18   float *y = (float*)malloc(N * sizeof(float));  
19  
20   for(int i=0; i<N; ++i)  
21   {  
22     x[i] = 2.0f;  
23     y[i] = 1.0f;  
24   }  
25  
26   saxpy(N, 3.0f, x, y);  
27  
28   return 0;  
29 }
```

saxpy.kernels.c

kernels構文のみ挿入

Compute region (kernels region)
→ GPU向けの“カーネル（主要な計算）”としてループが並列処理

CPU実行

GPU実行

CPU実行

コンパイル

```
$ nvc -acc -fast -Minfo=accel  
-gpu=cc80,cuda12.3 saxpy.kernels.c  
-o saxpy.exe
```

実行

```
$ ./saxpy.exe
```

NVIDIA HPC SDK利用例

CPU上でコンパイル・実行している様に扱える

まずはここから始めることを推奨

コンパイルと実行

注意: コンパイル・オプションの種類や指定方法はバージョンに依存します。

- 本資料: NVIDIA HPC SDK (旧PGIコンパイラ) を前提
 - ▶ オープン、先行事例多数、GPUを搭載するHPCIの計算機の多くで利用可能
 - ▶ <https://developer.nvidia.com/hpc-sdk>
- OpenACCを有効にする: `-acc`
 - ▶ `nvc/nvc++`, `nvfortran`に`-acc`を指定
- GPU/CPUの指定: `-acc`のサブオプションにて指定
 - ▶ `-acc=gpu` (デフォルト、gpu指定は省略可)
 - ▶ `-acc=multicore` (OpenMPと同様のマルチコアでのスレッド並列)
- GPU指定での詳細指定オプション (NVIDIA HPC SDK 23.11の場合)
 - ▶ `-gpu=cc80,cuda12.3,pinned` (NVIDIA A100向け、CUDA 12.3指定、pinnedメモリ使用)
 - ▶ `cc` (Compute Capability) の設定は`nvaccelinfo`コマンドで確認可能
- `-Minfo=accel`オプション (GPU実装関連のログ出力) は強く推奨
 - ▶ GPU実装をコンパイラに任せる以上確認はユーザの責任
- 実行
 - ▶ CPUでの実行と同様に操作すればOK

nvaccelinfoの出力例

CUDA Driver Version:	12020
...	
Device Name:	NVIDIA A100 80GB PCIe
...	
Default Target:	cc80

Minfo=accel指定時ログの例

■ データ転送・確保の情報

- ▶ 転送の位置（行番号）と方向、変数名、形状と要素数

■ GPU向け並列化の情報

- ▶ 並列化の位置（行番号）、種類（縮約計算など）
- ▶ 階層、粒度（発展）
- ▶ GPU向け並列に失敗した場合、その阻害要因

saxpy.kernels.c

```
3 void saxpy(int n,  
4           float a,  
5           float *x,  
6           float *restrict y)  
7 {  
8 #pragma acc kernels  
9   for(int i=0; i<n; ++i)  
10     y[i] = a*x[i]+y[i];  
11 }
```

data/loop構文相当の命令がコンパイラによって補完されていることが分かる

```
$ nvc -acc -fast -Minfo=accel -gpu=cc80,cuda12.3 saxpy.kernels.c
```

saxpy:

7, Generating implicit copy(y[:n])

Generating implicit copyin(x[:n])

9, Loop is parallelizable

Generating NVIDIA GPU code

9, #pragma acc loop gang, vector(128) /* blockIdx.x threadIdx.x

7行目にdata構文が挿入された ※節の意味は後述

implicit: コンパイラの判断による

データの転送位置（行番号）、変数名、転送の種類、範囲

9行目が並列化されてNVIDIA GPU向けコードが作成された

並列化の粒度の情報（発展）

kernels構文とparallel構文の比較

kernels

- 指示文はコンパイラへの「ヒント」
 - ▶ 並列化はコンパイラに任せる
- 並列化の可否 (データ依存性の有無など)はコンパイラが判断
 - ▶ parallelよりも慎重に並列化する
 - ▶ 並列化不可と判断したらシリアル処理扱いでコンパイルする

parallel

- 指示文はコンパイラへの一種の「命令」
 - ▶ kernels構文に比べ, ユーザが明示的な指定を行う
- 並列化の可否はユーザの判断
 - ▶ 並列化不可なループに適用すると誤った結果になる可能性

取扱注意!

両構文にはほぼ同様の節が用意されており、設定を細かく決めていくとほぼ同様に実行されることが殆どだが、基礎学習者はKernels構文を推奨（安全、簡単）

kernels構文での並列化

指定した領域内に含まれるループそれぞれを個別にGPUにオフロード

```
#pragma acc kernels  
{
```

ループそれぞれをオフロードする事を意識したシンプルな考え方

Loop 1 → GPUにオフロード
(GPU向け並列化、データ確保と転送、スレッドの設定)

Loop 2 → GPUにオフロード
(GPU向け並列化、データ確保と転送、スレッドの設定)

```
}
```

Loop1と2は別々にオフロードされる

parallel構文は指定した領域内に共通したスレッド群を設定し、loop構文で指定したループをスレッド群上にて並列化する (OpenMPの考え方と類似)

data構文・loop構文を明示的に指定する必要性

- 多くの場合、コンパイラがdata構文・loop構文に相当する命令を挿入する（Minfoオプションで確認できる）
 - ▶ ユーザによる明示的な指定を推奨
- 明示的な指定を推奨する理由
 - ▶ 正しく実行させる（コンパイラが処理を正しく理解できない）
 - ▶ 並列化に必要な情報を与える
 - ▶ 必要なデータを転送させる
 - ▶ 最適化
 - ▶ 冗長なデータ転送を排除する
 - ▶ 効率的な並列化を行う

明示的なdata構文の基本例

デバイス上での処理内容から必要な転送を判断

```
void saxpy(int n,  
           float a,  
           float *x,  
           float *restrict y)  
{  
    #pragma acc data copy(y[:n]), copyin(x[:n])  
    #pragma acc kernels  
    for(int i=0; i<n; ++i) {  
        y[i] = a*x[i] + y[i];  
    }  
}
```

CPU

GPU

備考: その他の
データ関連の構文
update
enter data
exit data

オフロードする領域
= Data構文が影響する領域

データ領域開始

データ領域終了

開始位置での動作:

1. y, xをデバイス (GPU) にて確保
2. y, xをホスト (CPU) からデバイスに転送

終了位置での動作:

1. yをデバイスからホストに転送
2. y, xをデバイスにて解放

デバイスでの確保・解放・転送を
ユーザが判断した上で、data構文、節にて
明示的に指定する

data節の例 (1/2)

備考: data構文において設定する際の挙動をベースにした説明であることに注意

■ データ領域でのデータ確保・転送・解放を節で指定

- ▶ copy (変数のリスト)
 - ▶ 領域の最初で確保とデバイスへ転送 + 最後でホストへ転送と解放
- ▶ copyin (変数のリスト)
 - ▶ 領域の最初で確保とデバイスへ転送 + 最後で解放
- ▶ copyout (変数のリスト)
 - ▶ 領域の最初で確保 + 最後でホストへ転送と解放
- ▶ create (変数のリスト)
 - ▶ 領域の最初で確保 + 最後で解放 (転送無しをコンパイラに伝える)
- ▶ present (変数のリスト)
 - ▶ 動作は無く、デバイスに確保されていることをコンパイラに伝えるのみ
 - ▶ 注意: presentで指定した変数で、実際にはデバイスに転送されていないときは実行時エラーとなる。

■ デバイスに確保済みの場合は、以上の命令はすべてpresentと同じ働き (動作無し) となる

- ▶ data構文を階層的に配置させる場合は慎重に行う必要あり

取扱注意！

data節の例 (2/2)

■ “変数のリスト”の指定方法

- ▶ copy節で例示 (copyin, copyout, create, presentで同じ)

- ▶ [設定] x: 1D配列 (N要素), y: 1D配列 (M要素), a: スカラー

- ▶ [C/C++] copy(x[0:N],y[0:M], a)

- ▶ “array[開始位置: 要素数]”で指定

- ▶ 開始位置を省略した場合は, 0が設定 (例: x[:N] = x[0:N])

- ▶ [Fortran] copy(x(1:N),y(1:M), a)

- ▶ “array(範囲の下限: 範囲の上限)”で指定

備考: 詳細 (多次元配列の一部など)は仕様書の”Data Specification in Data Clauses”を確認

- ▶ Q. 構造体は?

- ▶ A. 扱いが難しいため、利用を避けるのがbetter

- ▶ 参考: 第3講, <https://gdep-sol.co.jp/gpu-programming/openacc-no8/>

Data構文の明示的な入れ方

```
void saxpy(int n,  
          float a,  
          float *x,  
          float *restrict y)
```

```
{  
#pragma acc data copy(y[:n]), copyin(x[:n])  
#pragma acc kernels  
  for(int i=0; i<n; ++i)  
    y[i] = a*x[i] + y[i];  
}
```

計算の内容に基づいてコンパイラに頼らずにdata構文を挿入する

ユーザ指定のdata構文の内容がコンパイラの挿入する内容と重複する場合はユーザ指定が優先される

y, xをデバイスにて確保
y, xをデバイスに転送

yをホストに転送
y, xをデバイスにて解放

取扱注意！

- 挿入箇所（データ領域の範囲指定）、変数の指定と範囲はユーザが指示
 - ▶ Data構文の明示的な指定無しでのコンパイル時出力も参考に出来る

-Minfo=accelオプションの
メッセージ
(Data構文無指定時)

ver. 2.0.0.3

saxpy:

コンパイラが補完したdata構文 ユーザ指定に優先される

```
7, Generating implicit copy(y[:n]) [if not already present]  
Generating implicit copyin(x[:n]) [if not already present]
```

loop構文の役割

- 役割は基本 2 つ + 発展 1 つ
- 基本1: 並列化するループを指定する
 - ▶ parallel構文では各ループに最低 1 つ必要
- 基本2: 並列化を指示したループをコンパイラに正しく並列化させるための補足情報を与える
 - ▶ 適切な指示が無いと結果不正になる場合もある
 - ▶ 指示が抜けている時にコンパイラが警告を出してくれるとは限らない
- 発展: 並列化の階層割り当て
 - ▶ 基礎的な利用ではコンパイラに任せることを推奨
 - ▶ 第1講でGPUの階層構造について説明
 - ▶ ループ分割の粒度を設定してGPUの階層構造に対応させる
 - ▶ gang, worker, vectorの3階層

並列化の指示 (1/4)

■ 並列化する場所を指定する

```
#pragma acc kernels
{
    #pragma acc loop
    for(int i=0; i<n; ++i) {
        y[i] = a*x[i] + y[i];
    }
}
```

parallel構文内では、最低でも1つのループについてloop構文を指定すること。
指定無い場合は、逐次処理になる（非常に低速）

```
#pragma acc parallel
{
    #pragma acc loop
    for (i = 0; i < N; i++){
        C[i] = A[i] + B[i];
    }

    #pragma acc loop
    for (j = 0; j < N; j++){
        for (i = 0; i < N; i++){
            D[j][i] = E[j][i] *
F[j][i];
        }
    }
}
```

parallel構文内に複数ループを含む場合はそれぞれにloop構文を指定する

jループの並列化を指定している
iループについてはコンパイラの判断任せになる
(コンパイル時出力を確認)

並列化の指示 (2/4)

本来なら並列化出来るがコンパイラが並列化してくれないときに指定することが多い

■ independent節

- ▶ アドレスの競合が無いことを指示
 - ▶ C言語で必要とされること多い
 - ▶ ポインタにrestrict修飾子加えての対応も可能
- ▶ kernels構文（並列化に保守的）利用時に必要とされる事が多い
 - ▶ ループ内に「間接参照」の配列が存在する場合など

```
void func0(int n, float * a,  
           float * b)  
{  
    #pragma acc kernels  
    #pragma acc loop independent  
    for(int i=0; i<n; ++i)  
        a[i] = b[i];  
}
```

```
void func1(int n, float *restrict a,  
           float *restrict b, int *restrict ndx)  
{  
    #pragma acc kernels  
    #pragma acc loop independent  
    for(int i=0; i<n; ++i)  
        a[ndx[i]] = b[i];  
}
```

並列化の指示 (3/4)

■ private節

- ▶ 指定した変数をスレッド毎に確保させる (スレッド間で変数へのアクセスの干渉を防ぐ)
- ▶ 一時変数等のデータ競合を回避

58行目をコメントアウトした場合のコンパイル結果

```
57, Generating implicit copyout(tmp[:],A[:]) [if not already present]
59, Parallelization would require privatization of array tmp[:]
Generating Tesla code
59, #pragma acc loop seq
60, #pragma acc loop seq
64, #pragma acc loop seq
60, Loop is parallelizable
64, Loop is parallelizable
```

並列化出来ない
逐次 (sequential) 実行

cf. seq節: コンパイラによるループの
自動並列を抑制するときに指定する

```
56 #pragma acc kernels
57 {
58 #pragma acc loop private(tmp[:])
59     for (i = 0; i < N; i++) {
60         for (ii = 0; ii < 16; ii++) {
61             tmp[ii] = ii + i;
62         }
63         int sum = 0;
64         for (ii = 0; ii < 16; ii++) {
65             sum += tmp[ii];
66         }
67         A[i] = sum;
68     }
69 }
```

一時変数tmpをiイン
デックスそれぞれに
用意する必要がある

簡単なアルゴリズムの時は、
コンパイラが自動的に対応する

本例では一時配列tmpのprivate化
が要求されている

並列化の指示 (4/4)

■ reduction(operator : var-list)

- ▶ 各スレッドが同一変数に対して処理を行う
 - ▶ リダクション処理: 総和計算, 最大値の評価, etc.
 - ▶ コンパイラが補完してくれるときもあるが、明示的な指示を強く推奨

- parallel構文とloop構文で指定可能
- kernels構文では指定できない → kernels構文とloop構文の組み合わせで対応

```
while (err > thord) {  
    err = 0.0;  
  
    #pragma acc kernels  
    #pragma acc loop reduction(max:err)  
    for(int j = 0; j<N-1; ++j) {  
        for(int I = 0; i<N-1; ++i) {  
            A_upd[j][i] = (A[j][i+1] + A[j][i-1] +  
                           A[j-1][i] + A[j+1][i]) * 0.25;  
            err = fmax(err, fabs(A_upd[j][i] - A[j][i]));  
        }  
    }  
    // A_upd => A  
}
```

主な演算子と初期値 ()

+	加算	(0)
*	乗算	(1)
max	最大	(least)
min	最小	(largest)

並列化の階層割り当て (1/3)

■ 分割の3階層：gang, worker, vector

- ▶ loop構文の場合: gang(16), worker(1), vector(128)などで指定
 - ▶ 16x1x128並列化された場合に相当
- ▶ コンパイラが自動設定するがユーザ設定も可能
- ▶ kernels構文とparallel構文とで考え方に違いあり
 - ▶ parallel構文: 付随する構造化ブロック内でgang, worker, vectorの値は変更不可

■ 最適化が主な目的

- ▶ 後述するデータ転送の最適化に比べて難しい手法
 - ▶ GPUアーキテクチャのさらなる理解が必要
 - ▶ CUDAでのグリッド、ブロック、スレッド構成
 - ▶ GPUのSM (Streaming Multiprocessor) の占有率 (occupancy)など

並列化の階層割り当て (2/3)

■ コンパイラによる自動設定の例

```

40 #pragma acc kernels
41 {
42 #pragma acc loop
43     for (i = 0; i < N; i++)
44         C[i] = A[i] + B[i];
45 }
46
47 #pragma acc loop
48     for (j = 0; j < N; j++)
49         for (i = 0; i < N; i++)
50             D[j][i] = E[j][i] *
51             {
52             }
53 }

```

Loop構文を挿入したのみの例
分割の定量的な部分はコンパイラ任せ

gang分割、vector分割の場所と粒
度をコンパイラが設定している
ユーザが陽に設定することも可能

```

43, Loop is parallelizable
Generating Tesla code
43, #pragma acc loop gang, vector(128) /* blockIdx.x threadIdx.x */
48, Loop is parallelizable
49, Loop is parallelizable
Generating Tesla code
48, #pragma acc loop gang, vector(4) /* blockIdx.y threadIdx.y */
49, #pragma acc loop gang, vector(32) /* blockIdx.x threadIdx.x */

```

並列化の階層割り当て (3/3)

注意: あくまで実行のイメージ。
実際の動作とは異なる

■コンパイラによる自動設定の例: どのような実行か?

▶ N=1024でのイメージ (vectorの値で割り切れる場合)

```
43, Loop is parallelizable
Generating Tesla code
43, #pragma acc loop gang,
vector(128) /* blockIdx.x threadIdx.x
*/
```

```
40 #pragma acc kernels
41 {
42 #pragma acc loop
43   for (i = 0; i < N;
i++) {
44     C[i] = A[i] + B[i];
45   }
46
```

```
[C[0],...,C[127]]
= [A[0],...,A[127]]+[B[0],...,B[127]]
```

```
[C[128],...,C[255]]
= [A[128],...,A[255]]+[B[128],...,B[255]]
```

```
[C[256],...,C[383]]
= [A[256],...,A[383]]+[B[256],...,B[383]]
```

```
[C[384],...,C[511]]
= [A[384],...,A[511]]+[B[384],...,B[511]]
```

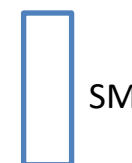
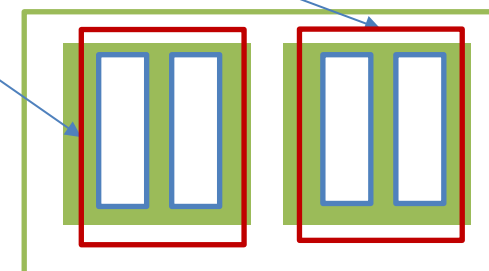
⋮

```
[C[896],...,C[1023]]
= [A[896],...,A[1023]]+[B[896],...,B[1023]]
```

考えやすさのため, 次のようなGPUを想定

- Streaming multiprocessor (SM)に64コア搭載
- SMを4基搭載

2 SMsで128スレッドを実行



gangの値はNに
よって変化

8(=N/128)分割
(8 gangs)

- 各カーネルのループはベクトル処理
- warp (32スレッド) 単位でまとめて処理

- 各gangが独立にGPUで実行されるカーネルに対応 (gang間で同期無し)
- コアが空いたところに, 未実行のgangに対応するカーネルが (次々と) 入る

OpenACCのデータ属性（data attribute）（1/4）

発展

- ここまでの説明をもとに、デバイスのスレッドからデータ（メモリ）がどのようにアクセスされるか整理する
- 基本：デバイスのメモリに転送/確保されたデータは、特定の指定がない限り、デバイスのスレッドからアクセス（読み込み/書き込み）ができる
- 特定の指定の例：private節
 - ▶ ユーザが陽に指定しない場合、暗黙的にコンパイラが設定する可能性もある
- 注意が必要な状況：異なるスレッドが同一メモリにアクセスし更新（書き込み）をする（データ競合/アクセス競合）
 - ▶ 例: 並列処理用のループインデックスに依存しない配列, スカラー変数
 - ▶ 更新される変数(式の左辺)に着目するとよい

OpenACCのデータ属性 (data attribute) (2/4)

発展

■ データ属性の種類

備考: データ属性については, OpenACC v3.3の規格書を参照することを推奨

■ predetermined: 既定の設定から変更できない

- ▶ forループ/doループのループインデックス: private節に指定された変数に相当
- ▶ [C/C++] ブロック内の一時変数: private節に指定された変数に相当

■ explicitly determined: data構文やCompute構文などにおけるdata節で陽に指定したデータ

- ▶ データ競合がある場合はprivate節などを設定する

■ implicitly determined: 上記2種以外、コンパイラが自動的に決定

- ▶ コンパイラが決める設定で問題があるときは、陽にdata節やprivate節などを設定する

OpenACCのデータ属性（data attribute）（3/4）

■ implicitly determinedの場合のスカラー変数

- ▶ kernels構文の場合：copy節が指定
- ▶ parallel構文の場合：firstprivate節が指定
 - ▶ private節と同じだが、初期値がparallel構文の直前における変数の値で設定
 - ▶ スカラー変数はスレッド間で干渉し合わないという挙動
- ▶ 読み込みのみのスカラー変数：implicitly determinedに基づく扱いで問題なし
 - ▶ 例：ループの回転数の属性はコンパイラ任せでよい
- ▶ 書き込みがあるスカラー変数：個別の設定が必要か検討が必要

OpenACCのデータ属性（data attribute）（4/4）

発展

■ データ競合がある場合への対処でよく使う節

- ▶ private節

- ▶ firstprivate節

- ▶ reduction節：指定した変数について並列リダクション演算適用

- ▶ 指定した変数 (リダクション変数)

- ▶ private節のようにスレッド固有の変数が生成される。

- ▶ スレッド固有の変数の値から評価された結果が指定した変数に格納される。

- ▶ 指定した変数には暗黙的にcopy節が設定される。

- ▶ 注意：parallel region内のloop構文や階層的並列を適用した状況において使用する場合は注意が必要

- ▶ 例：演算後の値の扱いについてprivate節が必要になる状況がある。

実践例：ヤコビ法の実装と最適化

■ 連立方程式の反復解法

- ▶ 広く利用されてきたもので、他に応用可能なアルゴリズム
- ▶ 格子点法で離散化
 - ▶ 周辺の値を規則的に足し込む事を収束するまで繰り返す、規則的な処理
- ▶ 高いデータ並列性をもつアルゴリズム
 - ▶ 格子点数でデータ並列化が可能
- ▶ データの局所性（デバイス上に保持）あり

■ OpenACCによるGPU実装と最適化の基本を学ぶ

- ▶ 3つの基本指示文の挿入
- ▶ プロファイリングの基本
 - ▶ データ取得、分析
- ▶ データ転送最適化の基本
 - ▶ OpenACC最適化で最も多く問題となるもの

ヤコビ法 反復法ソルバ

```
079: iconv = 0;
080: for ( itr = 1; itr <= MAXITR; ++itr) {
081:
082:     nrmsq = 0.0;
```

#define IDX(I,J) ((J) + (I)*(NY))

```
086:     for (int ix = 1; ix < NX-1; ++ix) {
087:         for (int iy = 1; iy < NY-1; ++iy) {
088:             phio[IDX(ix,iy)] = 0.25 * ( phie[IDX(ix+1,iy)] + phie[IDX(ix-1,iy)]
089:                                     + phie[IDX(ix,iy+1)] + phie[IDX(ix,iy-1)]
090:                                     + rho[IDX(ix,iy)] );
091:             nrmsq = fmax(nrmsq, fabs(phio[IDX(ix,iy)] - phie[IDX(ix,iy)]));
092:         }
093:     }
```

2次元領域でループ

次ステップ値
の計算

収束判定に用いる誤差の計算

```
108:     for ( int ix = 0; ix < NX; ++ix ) {
109:         for ( int iy = 0; iy < NY; ++iy ) {
110:             phie[IDX(ix,iy)] = phio[IDX(ix,iy)];
111:         }
112:     }
113:
114:     if ( tol*nrmsq >= nrmsq ) {
115:         iconv = 1;
116:         break;
117:     }
118: } /* itr */
```

次ステップ計算のために
配列をswapする

収束するまで反復

これまでに紹介してきた手法（指示文の挿入）
によるGPU向け実装方法を紹介する

ヤコビ法 反復法ソルバ

Data構文 (data) 、
Compute構文
(kernels) 、 Loop
構文 (loop) を挿入

```
079: iconv = 0;
080: for ( itr = 1; itr <= MAXITR; ++itr) { #define IDX(I,J) ( (J) + (I)*(NY) )
081:
082:     nrmsq = 0.0;
083:     #pragma acc data copyin(phie[0:NX*NY],rho[0:NX*NY]) copyout(phio[0:NX*NY])
084:     #pragma acc kernels
085:     #pragma acc loop independent collapse(2) reduction(max:nrmsq)
086:     for (int ix = 1; ix < NX-1; ++ix) {
087:         for (int iy = 1; iy < NY-1; ++iy) {
088:             phio[IDX(ix,iy)] = 0.25 * ( phie[IDX(ix+1,iy)] + phie[IDX(ix-1,iy)]
089:                                     + phie[IDX(ix,iy+1)] + phie[IDX(ix,iy-1)]
090:                                     + rho[IDX(ix,iy)] );
091:             nrmsq = fmax(nrmsq, fabs(phio[IDX(ix,iy)] - phie[IDX(ix,iy)]));
092:         }
093:     }
```

全てのスレッドで共有するスカラー値nrmsqにはreduction処理を指定

```
105: #pragma acc data copyin(phio[0:NX*NY]) copyout(phie[0:NX*NY])
106: #pragma acc kernels
107: #pragma acc loop independent collapse(2)
108: for ( int ix = 0; ix < NX; ++ix ) {
109:     for ( int iy = 0; iy < NY; ++iy ) {
110:         phie[IDX(ix,iy)] = phio[IDX(ix,iy)];
111:     }
112: }
113:
114: if ( tol*nrmsq >= nrmsq ) {
115:     iconv = 1;
116:     break;
117: }
118: } /* itr */
```

参照のみor更新有りに応
じてcopy/copyin/copyout
節を設定する

IDXによる参照でスレッド間でアドレス競合が無
いことをindependentを指定して保証する

ヤコビ法 反復法ソルバ

CPU

GPU

```
079: iconv = 0;
080: for ( itr = 1; itr <= MAXITR; ++itr) { #define IDX(I,J) ( (J) + (I)*(NY) )
081:
082: -- nrmsq = 0.0;
083: #pragma acc data copyin(phie[0:NX*NY],rho[0:NX*NY]) copyout(phio[0:NX*NY])
084: #pragma acc kernels
085: #pragma acc loop independent collapse(2) reduction(max:nrmsq)
086: for (int ix = 1; ix < NX-1; ++ix) {
087:   for (int iy = 1; iy < NY-1; ++iy) {
088:     phio[IDX(ix,iy)] = 0.25 * ( phie[IDX(ix+1,iy)] + phie[IDX(ix-1,iy)]
089:                               + phie[IDX(ix,iy+1)] + phie[IDX(ix,iy-1)]
090:                               + rho[IDX(ix,iy)] );
091:     nrmsq = fmax(nrmsq, fabs(phio[IDX(ix,iy)] - phie[IDX(ix,iy)]));
092:   }
093: }
...
105: #pragma acc data copyin(phio[0:NX*NY]) copyout(phie[0:NX*NY])
106: #pragma acc kernels
107: #pragma acc loop independent collapse(2)
108: for ( int ix = 0; ix < NX; ++ix ) {
109:   for ( int iy = 0; iy < NY; ++iy ) {
110:     phie[IDX(ix,iy)] = phio[IDX(ix,iy)];
111:   }
112: }
113:
114: if ( tol*nrmsq >= nrmsq ) {
115:   iconv = 1;
116:   break;
117: }
118: /* itr */
```

phie, rho

phio

phio

phie

コンパイルと実行

■ CPU向けコンパイル

▶ `nvc -acc=multicore -Minfo=accel -fast -tp=icelake jacobi.c -o jacobi.out`

■ GPU向けコンパイル

▶ `nvc -acc -Minfo=accel -gpu=cc80,cuda12.3,pinned -fast jacobi.c -o jacobi.out`

■ 同一コードからのコンパイルでCPUとGPUに対応

■ データサイズ: $n=8192$, $m=8192$ ($n \times m = 67108864$)

■ 実行環境

▶ CPU: Intel(R) Xeon(R) Platinum 8360Y CPU 2.4GHz, 36 cores

▶ GPU: NVIDIA A100 80GB PCIe

▶ NVIDIA HPC SDK 23.11, CUDA 12.3

コンパイルメッセージの確認

```
82, Generating copyout(phio[:67108864]) [if not already present]
```

```
Generating copyin(phie[:67108864]) [if not already present]
```

```
Generating implicit copy(nrmsq) [if not already present]
```

```
Generating copyin(rho[:67108864]) [if not already present]
```

明示的にdata構文を入れたので
Implicitの表示無し

明示的にdata構文で指定していない変数
(はimplicitで転送 (reduction変数のcopy))

```
86, Loop is parallelizable
```

```
87, Loop is parallelizable
```

```
Generating NVIDIA GPU code
```

```
86, #pragma acc loop gang, vector(128) collapse(2) /* blockIdx.x threadIdx.x */
```

```
Generating reduction(max:nrmsq)
```

```
87, /* blockIdx.x threadIdx.x collapsed */
```

reduction演算を指定、並列化

```
93, Generating copyin(phio[:67108864]) [if not already present]
```

```
Generating copyout(phie[:67108864]) [if not already present]
```

```
108, Loop is parallelizable
```

```
109, Loop is parallelizable
```

```
Generating NVIDIA GPU code
```

```
108, #pragma acc loop gang, vector(128) collapse(2) /* blockIdx.x threadIdx.x */
```

```
109, /* blockIdx.x threadIdx.x collapsed */
```

independentを指定したloop構文により並列化

指定した領域がGPUにオフロードされた

※正しくGPUへオフロードされたことの確認に加え、結果検証も重要

プロファイリング（性能評価）

■ NVIDIA Nsight Systems（nsys, nsys-ui）

▶ NVIDIA HPC SDKに含まれるプロファイラ

▶ nvprof, NVIDIA Visual Profiler (nvvp) は最新のGPUへの対応無く、移行推奨

▶ より詳しい説明は第3講を参照

■ テキスト（CLI）ベース：nsys（本講で扱う）

▶ 実行時に命令を加えるだけで利用は簡単

▶ 出力データが良くまとまった形で出力が可能、ボトルネックの特定向き

▶ GUIにオフラインで取り込む形式でも出力可能

■ GUIベース：nsys-ui（本講では扱い無し）

▶ GUI上で実行する又はnsys利用時の出力をGUIにオフラインで取り込む

▶ 詳細かつ多くの情報をGUI上に出力可能

▶ 時系列データの分析向き（ホストデバイス間転送、同期待ちの発生などの分析）

■ 「テキストで分析→GUIで詳細に分析」をお勧めします

■ （参考）NVIDIA Nsight Compute：GPUカーネル単位で詳細に分析するツール

▶ Nsight Systemsでの分析からさらに詳しく調べる場合に活用

テキストベースの実行例

```
$ nsys profile --stats=true ./jacobi.exe
```

性能比較

■ 反復計算のメインループの経過時間

- ▶ タイマー挿入で計測
- ▶ CPUは1 CPU（36スレッド並列）、GPUは1 GPUで実行
- ▶ 問題規模: 8192*8192, 反復は100回で強制打ち切り(未収束)

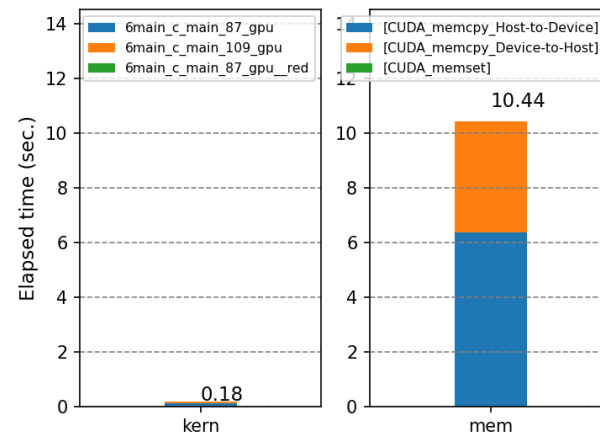
Type	Elapsed time of a main loop(sec.)	Speedup
CPU (acc=multicore)	1.97	1.0 (baseline)
GPU (acc=gpu)	10.6	0.19



nsysでの分析

■メモリ転送コストが大半を占める

```
nsys profile --stats=true ./jacobi.out 1> log.out 2>log.err
```



[6/8] Executing 'cuda_gpu_kern_sum' stats report

Time (%)	Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
57.7	105097840	100	1050978.4	1051345.5	1039297	1060641	4056.3	_6main_c_main_87_gpu
35.7	65115939	100	651159.4	651009.0	648577	653249	1041.3	_6main_c_main_109_gpu
6.6	11966603	100	119666.0	119568.0	117600	123104	1103.3	_6main_c_main_87_gpu_red

GPUカーネルのコスト

[7/8] Executing 'cuda_gpu_mem_time_sum' stats report

Time (%)	Total Time (ns)	Count	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Operation
61.0	6368024979	300	21226749.9	21225589.5	21210102	21304341	11429.6	[CUDA memcpy Host-to-Device]
39.0	4068691839	300	13562306.1	20337349.5	2751	20398293	9604029.5	[CUDA memcpy Device-to-Host]
0.0	105604	100	1056.0	1056.0	1023	1440	40.2	[CUDA memset]

メモリ転送・確保のコスト

ヤコビ法 反復法ソルバ

CPU

GPU

```
079: iconv = 0;
080: for ( itr = 1; itr <= MAXITR; ++itr) { #define IDX(I,J) ( (J) + (I)*(NY) )
081:
082: -- nrmsq = 0.0;
083: #pragma acc data copyin(phie[0:NX*NY],rho[0:NX*NY]) copyout(phio[0:NX*NY])
084: #pragma acc kernels
085: #pragma acc loop independent collapse(2) reduction(max:nrmsq)
086: for (int ix = 1; ix < NX-1; ++ix) {
087:   for (int iy = 1; iy < NY-1; ++iy) {
088:     phio[IDX(ix,iy)] = 0.25 * ( phie[IDX(ix+1,iy)] + phie[IDX(ix-1,iy)]
089:                               + phie[IDX(ix,iy+1)] + phie[IDX(ix,iy-1)]
090:                               + rho[IDX(ix,iy)] );
091:     nrmsq = fmax(nrmsq, fabs(phio[IDX(ix,iy)] - phie[IDX(ix,iy)]));
092:   }
093: }
...
105: #pragma acc data copyin(phio[0:NX*NY]) copyout(phie[0:NX*NY])
106: #pragma acc kernels
107: #pragma acc loop independent collapse(2)
108: for ( int ix = 0; ix < NX; ++ix ) {
109:   for ( int iy = 0; iy < NY; ++iy ) {
110:     phie[IDX(ix,iy)] = phio[IDX(ix,iy)];
111:   }
112: }
113:
114: if ( tol*nrmsq >= nrmsq ) {
115:   iconv = 1;
116:   break;
117: }
118: } /* itr */
```

無駄なデータ転送！
whileループの前後のみで十分

phie, rho

phie, rho

phio

phio

phie

phie

ヤコビ法 データ転送最適化版

CPU

GPU

phie, rho

```
078: #pragma acc data copyin(rho[0:NX*NY]) copy(phie[0:NX*NY]) create(phio[0:NX*NY]){
079:     iconv = 0;
080:     for ( itr = 1; itr <= MAXITR; ++itr) {
081:
082:         nrmsq = 0.0;
083:         #pragma acc data present(phie[0:NX*NY] rho[0:NX*NY])
084:         #pragma acc kernels
085:         #pragma acc loop independent collapse(2) reduction(max:nrmsq)
086:         for (int ix = 1; ix < NX-1; ++ix) {
087:             for (int iy = 1; iy < NY-1; ++iy) {
088:                 phio[IDX(ix,iy)] = 0.25 * ( phie[IDX(ix+1,iy)] + phie[IDX(ix-1,iy)]
089:                                     + phie[IDX(ix,iy+1)] + phie[IDX(ix,iy-1)]
090:                                     + rho[IDX(ix,iy)] );
091:                 nrmsq = fmax(nrmsq, fabs(phio[IDX(ix,iy)] - phie[IDX(ix,iy)]));
092:             }
093:         }
...
116:         #pragma acc data present(phie[0:NX*NY])
106:         #pragma acc kernels
107:         #pragma acc loop independent collapse(2)
108:         for ( int ix = 0; ix < NX; ++ix ) {
109:             for ( int iy = 0; iy < NY; ++iy ) {
110:                 phie[IDX(ix,iy)] = phio[IDX(ix,iy)];
111:             }
112:         }
113:
114:         if ( tol*nrmsq >= nrmsq ) {
115:             iconv = 1;
116:             break;
117:         }
118:     } /* itr */
```

ループ外で転送と確保

present節でデータの存在をコンパイラに教えて、ループ内での余分なデータ転送を排除

phie

性能比較

■ 反復計算のメインループの経過時間

- ▶ タイマー挿入で計測
- ▶ CPUは1 CPU（36スレッド並列）、GPUは1 GPUで実行
- ▶ 問題規模: 8192*8192, 反復は100回で強制打ち切り(未収束)

Type	Elapsed time of a main loop(sec.)	Speedup
CPU (acc=multicore)	1.97	1.0 (baseline)
GPU (acc=gpu)	10.6	0.19
GPU (acc=gpu,データ転送最適化版)	0.299	6.6



nsysでの分析

■メモリ転送コストの大幅な削減を確認

GPUカーネル関係 (演算)

[6/8] Executing 'cuda_gpu_kern_sum' stats report

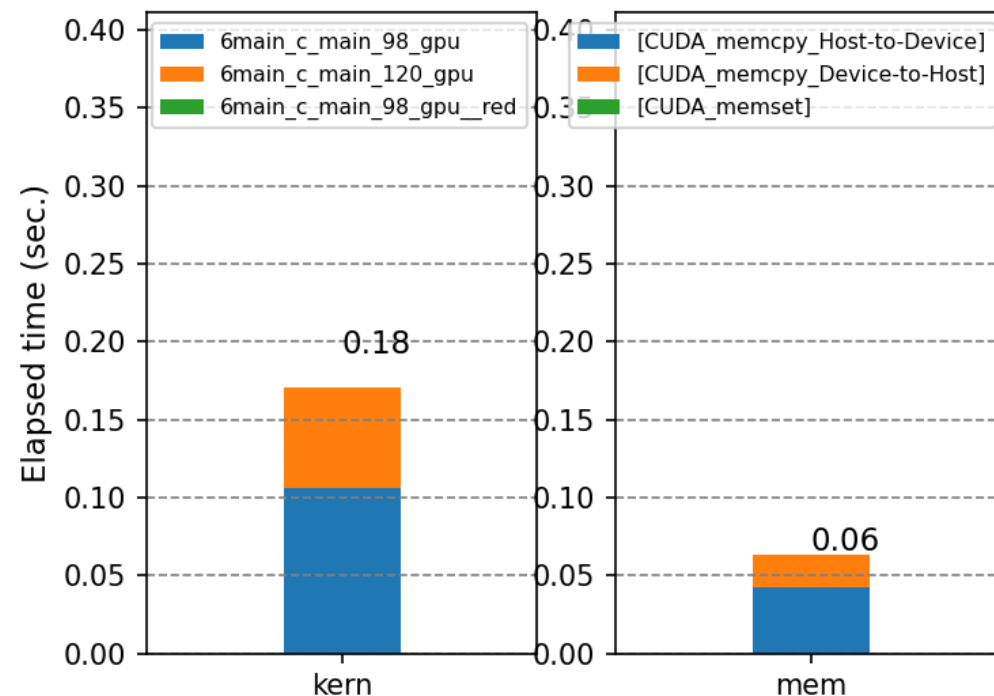
Time (%)	Total Time (ns)	Instances	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Name
58.2	106481705	100	1064817.1	1065092.0	1055875	1074308	3288.1	_6main_c_main_98_gpu
35.2	64445604	100	644456.0	644418.0	642114	646594	975.6	_6main_c_main_120_gpu
6.5	11965256	100	119652.6	119632.0	117408	123200	1091.9	_6main_c_main_98_gpu_red

[7/8] Executing 'cuda_gpu_mem_time_sum' stats report

Time (%)	Total Time (ns)	Count	Avg (ns)	Med (ns)	Min (ns)	Max (ns)	StdDev (ns)	Operation
67.4	42515746	2	21257873.0	21257873.0	21237761	21277985	28442.7	[CUDA memcpy Host-to-Device]
32.5	20479135	101	202763.7	1440.0	1407	20332862	2023049.9	[CUDA memcpy Device-to-Host]
0.2	105702	100	1057.0	1056.0	1024	1440	39.8	[CUDA memset]

HtoD 63.680s -> 42.516ms
DtoH 40.687s -> 20.479ms

メモリ転送関係



[参考] 性能比較：NVIDIA Tesla V100の場合

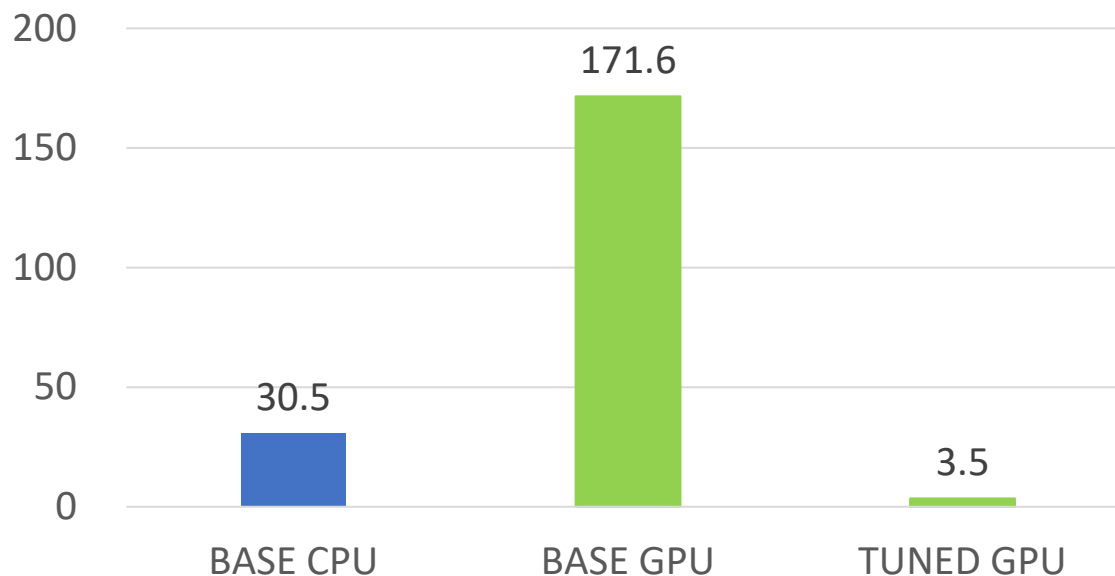
■ 経過時間の比較

▶ CPU (acc=multicore) : 30.5[s]

▶ GPU (acc=gpu) : 171.6[s]

▶ GPU (acc=gpu, 最適化版) : 3.5[s]

▶ CPUは1CPU (20コア)、GPUは1GPUで実行



注意

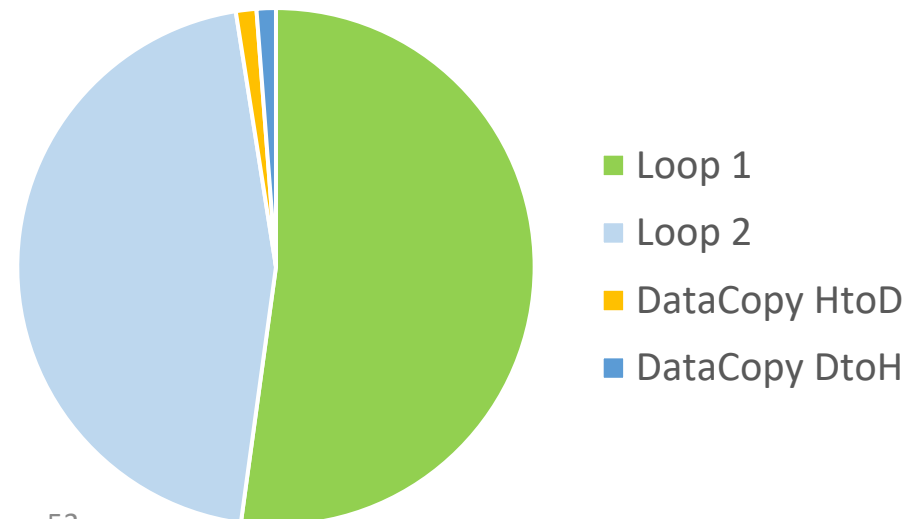
- A100における計測で使用したコードと実装が少し異なる (本質的には同じ)
- A100における計測と反復回数が異なる。
- 問題規模: 8192*8192
- 設定
 - CPU: Intel Xeon Gold 6210U 2.5GHz 20cores
 - GPU: NVIDIA Tesla V100
 - NVIDIA HPC SDK 21.9, CUDA 11.4
 - CPU向け: `nvc -acc=multicore -Minfo=accel -fast -tp=haswell ...`
 - GPU向け: `nvc -acc -Minfo=accel -gpu=cc70,cuda11.4,pinned ...`

[参考] nsysでの分析：NVIDIA Tesla V100の場合

■メモリ転送コストの大幅な削減を確認

Time(%)	Time	Calls	Avg	Min	Max	Name
49.14%	1.70518s	1000	1.7052ms	1.6822ms	1.7327ms	main_63_gpu
45.34%	1.57345s	1000	1.5735ms	1.5538ms	1.5904ms	main_74_gpu
3.03%	105.16ms	1000	105.16us	102.30us	108.22us	main_63_gpu__red
1.28%	44.507ms	1001	44.462us	1.0240us	43.428ms	[CUDA memcpy HtoD]
1.21%	41.850ms	1001	41.808us	1.1830us	40.636ms	[CUDA memcpy DtoH]

HtoD 86.8427s -> 44.507ms
DtoH 81.2974s -> 41.850ms



OpenACCでのGPU実装方法 まとめ

1. 最初は小さくかつ処理を把握し易いループへのkernels構文適用を試みる
 - -Minfo=accelの設定で、GPU向け並列化が意図したとおりに行われているか確認
 - 並列化されない場合は適切なloop構文を挿入するか、アルゴリズムを見直す
 - 既にOpenMPにて並列化されたループは、そのままkernels構文に置き換える (or 併用させる)
2. コンパイラが設定したdata構文の内容 (挿入位置、配列名と配列の大きさ) を確認
 - 必要に応じてdata構文を用いて配列の要素数と形状を指定
3. 結果が正しいか、CPUでの実行と差異がある場合はそれは十分小さいかを確認する
4. 実行性能を取得、分析する (nsysなど活用)
5. data構文を用いてホストデバイス転送の最適化を行う

CPUコードからの移行 注意点

- CPUコードへのOpenACC指示文の挿入・データ転送最適化だけでは、GPUで実行は正常にできて性能が出ない場合もある
- 注意すべきこと
 - ▶ 並列数（ループの回転数）が性能を出すには足りない場合がある
 - ▶ アルゴリズムを見直して並列性を抽出
 - ▶ loop構文を活用した最適化 (例: collapse節の指定して回転数を増やす)
 - ▶ ストライドアクセスが原因で（CPUはキャッシュでカバーできている）GPUでは性能が出ない場合がある
 - ▶ データ構造、ループ構造の見直し
 - ▶ CPUコードの情報を十分に把握する
 - ▶ (可能なら並列計算のときの) 性能
 - ▶ 典型的な実行時のメモリ使用量

第2講のまとめ

- OpenACCを用いてアプリケーションをGPUに実装し、最適化するための基礎を紹介しました
- OpenACCは、GPU向けの並列化とホストデバイス間でのメモリ管理という、GPUコンピューティングで必須な要素を効率よく実装出来るプログラミングモデルです
- OpenACCの基本の構文と、実践例としてヤコビ法の実装・性能分析・最適化の基礎を紹介しました

補足:説明しなかった構文,指示文,節 (1/13)

- 本編で詳細な説明をしなかった構文, 指示文, 節の中で、よく使うと考えられるものを簡単に紹介します。
 - ▶ Full reference guideではありません。
 - ▶ 第3講で一部を事例として紹介しています。
 - ▶ 網羅的なまとめとして、OpenACCのAPI Reference Cardを参照することを推奨します。
 - ▶ <https://www.openacc.org/resources>

補足:説明しなかった構文,指示文,節 (2/13)

- seq節: ループ反復の添字についてデータ(配列)に依存性があることを指示。コンパイラは逐次計算の命令を生成

```
#pragma acc kernels
{
#pragma acc loop independent
for (i=0;i<SIZE;++i) {
#pragma acc loop independent
for (j=0; j<SIZE;++j) {
#pragma acc loop seq
for (k=0; k<SIZE;++k) {
c[i][j] += a[i][k] * b[k][j]
}
}
}
} /* acc kernels */
```

C

```
!$ACC kernels

!$ACC loop independent
do j = 1, SIZE
!$ACC loop independent
do i = 1, SIZE
!$ACC loop seq
do k=1, SIZE
c(i,j) = c(i,j) + a(i,k) * b(k,j)
end do
end do
end do

!$ACC end kernels
```

Fortran

補足:説明しなかった構文,指示文,節 (3/13)

■ collapse節: 指示文直下において、指定したレベルまでの入れ子ループをまとめて並列化

- ▶ 効果1: ループごとに指示文を書かずに済むため楽ができる
- ▶ 効果2: 短いループをまとめることで粒度を上げ, 並列化の効果を出す

```
#pragma acc kernels
{
#pragma acc loop collapse(2) independent
for (i=0;i<SIZE;++i) {
for (j=0; j<SIZE;++j) {
#pragma acc loop seq
for (k=0; k<SIZE;++k) {
c[i][j] += a[i][k] * b[k][j]
}
}}
} /* acc kernels */
```

C

```
!$ACC kernels

!$ACC loop collapse(2) independent
do j = 1, SIZE
do i = 1, SIZE
!$ACC loop seq
do k=1, SIZE
c(i,j) = c(i,j) + a(i,k) * b(k,j)
end do
end do
end do

!$ACC end kernels
```

Fortran

補足:説明しなかった構文,指示文,節 (4/13)

- 非同期処理で使用する節と指示文 (enter data指示文/exit data指示文、および第3講を参照)
 - ▶ **async節: async(n)**
 - ▶ 計算領域の末端の暗黙の同期やデータ転送の同期を無効にする。()に管理IDを指定する (例: async(0))。
 - ▶ **wait節: wait(n)**
 - ▶ 指定した計算領域やデータ転送の実行を待機状態にする。実行開始はasync節で指定したIDで管理する (例: wait(1)はasync(1)で指定した処理完了まで待つ)。
 - ▶ **wait指示文: [C/C++] #pragma acc wait (n) / [Fortran] !\$ACC wait(n)**
 - ▶ プログラムの任意の場所で処理の同期ポイントを設定。wait節と同様にasync節で指定したIDを設定する。

補足:説明しなかった構文,指示文,節 (5/13)

■ enter data指示文/exit data指示文 (1/3)

- ▶ data構文の機能を入口側 (enter)と出口側(exit)とに分離した構文
 - ▶ enter data: ホストからデバイスに転送するデータを指定
 - ▶ `copyin`節, `create`節を主に設定する
 - ▶ exit data: デバイスからホストに転送するデータを指定
 - ▶ `copyout`節, `delete`節を主に設定する
- ▶ 主な利用場面 (第3講で事例を紹介)
 - ▶ 非同期処理の適用で利用する (データ転送とCPUの計算を重ねる)
 - ▶ 備考: data構文では`async`節や`wait`節の設定はできない
 - ▶ 柔軟にデータ転送を設計する
 - ▶ クラス (C++)のコンストラクタ/デストラクタと組み合わせる

補足:説明しなかった構文,指示文,節 (6/13)

■ enter data指示文/exit data指示文 (2/3)

▶ 例1: data構文の書き換え (機能としては等価)

▶ 注意: enter data指示文にcopy節の設定はできない

```
#pragma acc enter data copyin(A[0:N],B[0:N])
```

C

```
#pragma acc kernels loop independent
for (i=0;i<N;++i) {
    A[i] = A[i] + B[i];
}
```

```
#pragma acc exit data copyout(A[0:N])
```

=

```
#pragma acc data copyin(B[0:N]) copy(A[0:N])
{
#pragma acc kernels loop independent
for (i=0;i<N;++i) {
    A[i] = A[i] + B[i];
}
} /* acc data */
```

```
!$ACC enter data copyin(A(1:N),B(1:N))
```

Fortran

```
!$ACC kernels loop independent
do i = 1, N
    A(i) = A(i) + B(i)
end do
!$ACC end kernels loop
```

```
!$ACC exit data copyout(A(1:N))
```

=

```
!$ACC data copyin(B(1:N)) copy(A(1:N))

!$ACC kernels loop independent
do i = 1, N
    A(i) = A(i) + B(i)
end do
!$ACC end kernels loop

!$ACC end data
```

補足:説明しなかった構文,指示文,節 (7/13)

■ enter data指示文/exit data指示文 (3/3)

▶ 例2: 非同期処理の例 (第3講を参照)

```
/* Bを非同期で転送 */
#pragma acc enter data copyin(B[0:N]) \
    create(A[0:N]) async(0)

do_work_in_host(C); /* ホストで計算 */
/* Bの転送完了まで待機。計算(カーネル)を非同期に実行 */
#pragma acc kernels async(1) wait(0)
#pragma acc loop independent
for (i=0;i<N;++i) {
    A[i] = sin(u*B[i]);
}
/* 計算(カーネル)完了まで待機。Aを非同期で転送 */
#pragma acc exit data copyout(A[0:N]) async(2) wait(1)

do_work_in_host(D); /* ホストで計算 */

#pragma acc wait(2) /* Aの転送完了まで待機 */
```

C

```
! Bを非同期で転送
!$ACC enter data copyin(B(1:N)) &
!$ACC & create(A(1:N)) async(0)

call do_work_in_host(C) ! ホストで計算
! Bの転送完了まで待機。計算(カーネル)を非同期に実行
!$ACC kernels async(1) wait(0)
!$ACC loop independent
do i = 1, N
    A(i) = sin(u*B(i))
end do
!$ACC end kernels
! 計算(カーネル)完了まで待機。Aを非同期で転送
!$ACC exit data copyout(A(1:N)) async(2) wait(1)

Call do_work_in_host(D) ! ホストで計算

!$ACC wait(2) ! Aの転送完了まで待機
```

Fortran

補足:説明しなかった構文,指示文,節 (8/13)

■ update指示文

- ▶ ホストとデバイスのメモリ間でデータをコピーする。任意のタイミングでコピーが可能。
- ▶ 節で動作が変わる。
 - ▶ `update self(変数)`: 変数について、デバイスからホストにコピー
 - ▶ `self`は`host`と書くことも可能
 - ▶ `update device(変数)`: 変数について、ホストからデバイスにコピー
- ▶ `async`節の設定も可能。(非同期でのデータ転送)
- ▶ 第3講で事例を紹介
 - ▶ MPI通信関数へのデータ受け渡し
 - ▶ C++クラスにおける事例

補足:説明しなかった構文,指示文,節 (9/13)

■ host_data構文 + use_device節

- ▶ デバイス上メモリにあるデータ (のアドレス) をホスト側で利用可能にする。
 - ▶ host_data use_device(変数名): 変数名を指定すること。
- ▶ 典型的な利用例: OpenACCの指示文を使ってデバイス上に置いたデータを、(CUDAで記述された)ライブラリの関数にわたす。
- ▶ 第3講で事例を紹介
 - ▶ CUDAで記述された関数やcuBLASにデータを渡す。
 - ▶ CUDA-aware MPIライブラリのMPI通信関数にデータを渡す。

補足:説明しなかった構文,指示文,節 (10/13)

■ declare指示文

- ▶ Fortranのmoduleで宣言された変数やC/C++のグローバルなスコープをもつ変数などについて、プログラム実行時にデバイス上のメモリを確保することを指定する。
- ▶ 節でメモリ確保を指定する。data節は全て指定可能
 - ▶ declare create(変数): data構文のcreate節と同じ。ホスト上でも変数を使用する場合に指定すると覚えるのがよい。
 - ▶ declare device_resident(変数): デバイス上でのみ使用する変数のときに指定すると覚えるのがよい。
- ▶ 第3講で事例を紹介
 - ▶ Fortranのmodule内で宣言した変数の利用 (手動deep copyの実装例)

補足:説明しなかった構文,指示文,節 (11/13)

■ routine指示文 (1/3)

- ▶ parallel領域で呼び出す関数について、デバイス上で実行することを指定する。
- ▶ 関数の宣言時に指示文を指定する。その際に、gang, worker, vector, seqの少なくとも一つを節として設定する。
 - ▶ gang: 関数の呼び出し先の並列階層がgangの場合
 - ▶ 関数内のループをgang, worker, vectorで並列化できる
 - ▶ worker: 関数の呼び出し先の並列階層がworkerの場合
 - ▶ 関数内のループをworker, vectorで並列化できる
 - ▶ vector: 関数の呼び出し先の並列階層がvectorの場合
 - ▶ 関数内のループをvectorでのみ並列化できる。
 - ▶ seq: 関数の呼び出し先で並列計算を行っていない場合
 - ▶ 関数内のループの実行はseq (逐次的) のみ。並列実行はしない。

補足:説明しなかった構文,指示文,節 (12/13)

■ routine指示文 (2/3)

▶ 例

```
#pragma acc routine gang
void calc_device (double * a, double * b, const int ns)
{
    #pragma acc loop independent
    for (int i=0; i<ns; ++i) {
        a[i] = a[i] + b[i];
    }
}
...

#pragma acc data copyin(b[0:ns]) copyout(a[0:ns])
{
    #pragma acc parallel present(a[0:ns],b[0:ns])
    {
        calc_device(a, b, ns);
    }
}
```

C

```
subroutine calc_device (a, b, ns)
!$ACC routine gang
    real(kind=DP),contiguous,intent(inout) :: a(:)
    real(kind=DP),contiguous,intent(in) :: b(:)
    integer,intent(in) :: ns
    integer :: i
    !$ACC loop
    do i = 1, ns
        a(i) = a(i) + b(i)
    end do
end subroutine calc_device
...

!$ACC data copyin(b(1:ns)) copyout(a(1:ns))
!$ACC parallel present(a(1:ns),b(1:ns))
call calc_device(a, b, ns)
!$ACC end parallel
!$ACC end data
```

Fortran

補足:説明しなかった構文,指示文,節 (13/13)

■ routine指示文 (3/3)

▶ 留意事項

- ▶ まずはroutine指示文を使用せずに対応することを検討する。

- ▶ 例: 小さな関数であればインライン展開することで使用を回避

▶ 理由

- ▶ 階層的並列について正確に理解する必要がある。
 - ▶ 関数コールのオーバヘッドが性能へ顕著な影響を与えることがある。
 - ▶ 関数内の一時データ (デバイス上の”スタック”に対応) の肥大化。予期せずメモリ不足に陥ることがある。
 - ▶ <https://forums.developer.nvidia.com/t/what-is-the-maximum-cuda-stack-frame-size-per-kerenl/31449>

補足: Runtime API routine

- 指示行の指定ではできない操作を可能
 - ▶ C/C++: #include "openacc.h" (extern "C"で記述)
 - ▶ Fortran: use openacc
- (大別して) 4種類の機能が提供
 - ▶ デバイスの管理と情報取得 (acc_get_num_devicesなど)
 - ▶ 非同期処理の管理 (acc_async_waitなど)
 - ▶ メモリ管理 (デバイス側のメモリーを割り当て; acc_malloc, acc_free)
 - ▶ ランタイムの初期化
- 基本的にはホスト側のみで利用可能な関数 (parallel/kernels領域の外側)
 - ▶ acc_on_deviceはデバイス側で利用可能

補足: 階層的並列に関する用語

[粒度] = gang, worker, vector

■ OpenACCの規格書を読む場合、把握しておくといよい用語

- ▶ [粒度]-redundantモード: [粒度]について並列に同じ計算を実行
 - ▶ gang-redundantモード: 並列領域実行の開始に対応
- ▶ [粒度]-partitionedモード: [粒度]について並列に異なる(分割された)計算を実行
 - ▶ gang-partitionedモード: gangについて分割
 - ▶ worker-partitionedモード: 分割された各gang内で workerについて分割
 - ▶ vector-partitionedモード: 分割された各gangに含まれる分割された各worker内で vectorについて分割
- ▶ [粒度]-singleモード: 1個の[粒度]だけがアクティブな状態
 - ▶ worker-singleモード
 - ▶ vector-singleモード

補足: OpenACCとOpenMPの比較 (1/7)

■ OpenMP

- ▶ OpenACCと同様に、指示文ベースでコンパイラにまかせてGPU向けコードをプログラミングできる。
- ▶ OpenMPにおけるデバイス向け (ex. GPGPU) 構文: Device constructで定められている。
- ▶ v5.0規格で拡充。

■ ここでは

- ▶ OpenACCを学習した方に向けて、OpenMP (v5.0, v5.2)と比較する。
 - ▶ 注意: OpenMP v6.0で一部の指示文の名称が変更。v5.0-v5.2との互換性はある模様だが詳細は規格書の確認が必要。
- ▶ 基礎的な内容 (第2講の内容)については「読み替え」しやすい(と思う)。
 - ▶ (参考) Solomon: Simple Off-Loading Macros Orchestrating multiple Notations;
<https://github.com/ymiki-repo/Solomon>

■ 詳しくは

- ▶ T. Deakin and T. G. Mattson, “Programming your GPU with OpenMP: Performance Portability for GPUs” (2023, MIT)

補足: OpenACCとOpenMPの比較 (2/7)

■ OpenACCとOpenMP: 開発環境

▶ 例: NVIDIA HPC SDK (<https://docs.nvidia.com/hpc-sdk/compiler/index.html>)

▶ OpenACCによるGPU利用, OpenMPによるGPU利用のどちらも対応可能

▶ コンパイルの方法 (NVIDIA HPC SDK 23.11, CUDA 12.0, Compute Capability 8.0)

▶ OpenMP [C] : `nvc -mp=gpu -gpu=cc80,cuda12.0 -Minfo=mp,accel (...)`

▶ OpenACC [C] : `nvc -acc=gpu -gpu=cc80,cuda12.0 -Minfo=accel (...)`

▶ OpenMP [C++] : `nvc++ -mp=gpu -gpu=cc80,cuda12.0 -Minfo=mp,accel (...)`

▶ OpenACC [C++] : `nvc++ -acc=gpu -gpu=cc80,cuda12.0 -Minfo=accel (...)`

▶ OpenMP [Fortran] : `nvfortran -mp=gpu -gpu=cc80,cuda12.0 -Minfo=mp,accel (...)`

▶ OpenACC [Fortran] : `nvfortran -acc=gpu -gpu=cc80,cuda12.0 -Minfo=accel (...)`

▶ 応用: OpenMPでCPUスレッド並列, OpenACCでGPU利用

`nvc -mp=multicore -acc=gpu -gpu=cc80,cuda12.0 -tp=native -Minfo=mp,accel (...)`

補足: OpenACCとOpenMPの比較 (3/7)

■ はじめてGPUプログラミングを行う場合の方法

- ▶ OpenACC: **kernels**構文と**loop**構文の組み合わせを使う。
- ▶ OpenMP: **target**構文と**loop**構文の組み合わせを使う。
- ▶ 共通: 階層的並列は無理に利用しない。

OpenACC

```
#pragma acc kernels
#pramga acc loop
for (i=0; i<NS; ++i) {
    y[i] += a*x[i]:
}
```

OpenMP

```
#pragma omp target
#pragma omp loop
for (i=0; i<NS; ++i) {
    y[i] += a*x[i]:
}
```

補足: OpenACCとOpenMPの比較 (4/7)

■ ホスト-デバイス間のデータ転送の方法

- ▶ OpenACC: **data**構文において節 (**copy**, **copyin**, **copyout**, **present**, **create**)を指定して制御する。
- ▶ OpenMP: **target data**構文において節+modifier (**map(tofrom:)**, **map(to:)**, **map(from:)**, **map(present:)**, **map(alloc:)**)を指定して制御する。

OpenACC

```
#pragma acc data copyin(x[0:NS]) \
copyout(y[0:NS])
{
    #pragma acc kernels present(x,y)
    #pramga acc loop
    for (i=0; i<NSIZE; ++i) {
        y[i] += a*x[i];
    }
}
```

OpenMP

```
#pragma omp target data map(to:x[0:NS]) \
map(from:y[0:NS])
{
    #pragma omp target map(present:x,y)
    #pragma omp loop
    for (i=0; i<NSIZE; ++i) {
        y[i] += a*x[i];
    }
}
```

備考: OpenMP v6.0ではtarget_data構文。
target dataも引き続き利用可能(な模様)

補足: OpenACCとOpenMPの比較 (5/7)

■ (発展) CUDAで記述された関数をホスト側で呼び出すときにデータ (デバイス・ポインタ) を渡す方法 (第3講を参照)

- ▶ OpenACC: `host_data`構文において`use_device`節で指定する。
- ▶ OpenMP: `target data`構文において`use_device_ptr`節で指定する。

OpenACC

```
#pragma acc data copyin(x[0:NS]) \
copyout(y[0:NS])
{
    #pragma acc host_data use_device(x,y)
    {
        my_saxpy_cuda(NS, x, y);
    }
    ...
}
```

OpenMP

```
#pragma omp target data map(to:x[0:NS]) \
map(from:y[0:NS])
{
    #pragma omp target data use_device_ptr(x,y)
    {
        my_saxpy_cuda(NS, x, y);
    }
    ...
}
```

備考: OpenMP v6.0では`target_data`構文。
`target data`も引き続き利用可能(な模様)

補足: OpenACCとOpenMPの比較 (6/7)

■ (発展) 階層的な並列自由度の比較 → 並列化の粒度と関連

▶ GPGPUの場合: 基本は2階層

▶ 大きな粒度: grid (CUDA*), NDRange (OpenCL**)

▶ 小さな粒度: thread block (CUDA), work-group (OpenCL)

*: <https://docs.nvidia.com/cuda/index.html>

** : <https://www.khronos.org/registry/OpenCL/>

▶ 対応する階層

▶ OpenACC

▶ 大きな粒度: gang

▶ loop構文 + gang指示節で対応するループを並列化

▶ 小さな粒度: worker, vector

▶ loop構文 + (worker指示節, vector指示節)で対応するループを並列化

▶ OpenMP

▶ 大きな粒度: League

▶ target構文中のteams構文で生成; distribute構文で対応するループを並列化

▶ target構文中にteams構文は複数書くことはできない (Leagueの生成は1回だけ)

▶ 小さな粒度: Thread team

▶ parallel構文で生成; parallel for, parallel for simdで対応するループを並列化

補足: OpenACCとOpenMPの比較 (7/7)

■(発展) 非同期処理

- ▶ OpenACC: `async`節と`wait`節で制御する。
 - ▶ 計算: `kernels`構文や`parallel`構文で指定する。
 - ▶ データ転送: `enter data`指示文, `exit data`指示文などで指定する。
- ▶ OpenMP: `nowait`節と`depend`節+modifier (`depend(in:)`, `depend(out:)`等)で制御する。
 - ▶ 計算: `target`構文で指定する。
 - ▶ データ転送: `target enter data`構文, `target exit data`構文などで指定する。

備考: OpenMP v6.0では`target_data`構文, `target_enter_data`構文, `target_exit_data`構文。以前の記述も引き続き利用可能(な模様)

謝辞

- 本資料 (サンプルコード, 計測) の一部において、東京科学大学のスーパーコンピュータTSUBAME4.0を利用しました。

参考文献

1. HPC WORLD 「OpenACCによるプログラミング」, <https://hpcworld.jp/archive/SPG/Pgi/OpenACC/>
2. HPC WORLD: HPC技術コラム, <https://hpcworld.jp/techcolumn/>
3. GDEP, GPUコラム 「OpenACCではじめるGPUプログラミング」, <https://gdep-sol.co.jp/gpu-programming/>
4. 株式会社Metro, 「GPU高速化事例」, <https://www.metro.co.jp/casestudy/>
5. Rob Farber, “Parallel Programming with OpenACC” (MK, 2016).
6. John Cheng, Max Grossman, Ty McKercher (訳: 株式会社クイープ, 監訳: 森野慎也, 成瀬彰), 「CUDA C プロフェッショナルプログラミング」 (インプレス, 2015) 第8章.
7. OpenACC Getting Started Guide, <https://docs.nvidia.com/hpc-sdk/compiler/openacc-gs/index.html>
8. NVIDIA, OpenACC Overview Course, <https://developer.nvidia.com/openacc-overview-course>
9. OpenACC Specification, <https://www.openacc.org/specification>

本資料は、構成・文章・画像などの全てにおいて著作権法上の保護を受けています。以下の条件で本資料の利用を許可します。

- 本資料の利用に際し、著作者への連絡は不要ですが、著作者のクレジット（当機構の名称）は明示して下さい。
- 本資料の一部あるいは全部について、転載、複写及び再配布を許可します。ただし、本資料を改変した場合は除きます。
- 本資料を販売するなどの営利目的での利用は禁じます。教育目的等で利用することは産・学・官を問わず許可します。

※上記はクリエイティブ・コモンズ「表示-非営利-改変禁止 4.0 国際（CC BY-NC-ND 4.0）」相当です。

- 本資料に記載された内容などは、予告なく変更される場合があります。
- 本資料に起因して使用者に直接または間接的損害が生じても、著作者はいかなる責任も負わないものとします。

なお、本資料の旧版についても上記と同様の条件で利用することを許可します。

ご不明の点があれば下記のヘルプデスクにお問い合わせ下さい。

ヘルプデスク： helpdesk[-at-]hpci-office.jp（[-at-]を@にしてください）

